

Auditable Trace Privacy with Abort: Disclosure receipts for side-channel resistant transport

February 27, 2026

Abstract

Even if a system design satisfies trace-privacy theorems on paper, real deployments drift: configuration changes, partial rollouts, shared pools, and “innocent” performance fixes can silently reintroduce trace leakage. We propose an operational layer for trace privacy built around *proof-carrying plans*: (1) every deployment binds itself to a public schedule skeleton (an SRPplan) via a certificate-like commitment, (2) records that commitment in a transparency log, and (3) continuously monitors traces against the plan using *anytime-valid* statistical evidence (e-values / e-processes). The result is a practical regime of (i) **fast auditing** (verify what was intended), (ii) **split-view risk reduction** (public log consistency plus witnesses/gossip), and (iii) **continuous regression detection** (safe under optional stopping). We provide toy-but-executable tooling: plan certificates, a plan transparency log (PTL), and a trace-audit CLI.

Scope. We isolate and formalize *Auditable Trace Privacy with Abort: Disclosure receipts for side-channel resistant transport* as a reusable contract surface in the anonymity / Anonymous-DHT series.

Units. Budgets are published in bits (use $\log_2$ and 2^ϵ); results stated in nats can be converted by dividing by $\ln 2$ (see Synthesis 8).[63]

Imports. We treat stop-time padding and the compiler/scheduling machinery as imported from the backbone posts.[61, 62] This operational paper specializes that machinery to the retry surface and to schedule-only deployments. We also treat beacon/VRF binding and append-only log publication as imported primitives (Synthesis 9).[54] We import interface and threat/window identifiers (ENF-ID and TW-ID) so trace and plan objects can be release-pinned without prose drift.[55, 56]

Artifact policy. This is a source-only release. Supporting artifacts (logs, traces, scripts, certificates, and verifier outputs) are available on request as a single bundle, committed by ABOM/OINL (Anonymity C) and governed by the series artifact policy (Synthesis 6).[59]

Exports. Exports an auditable trace-privacy contract: (i) a trace schema, (ii) a redaction function defining the public view, and (iii) a monitoring receipt certifying an epoch’s leakage budget. Deployments additionally export `plan_id` and digest-bound `plan_spec_id` (Synthesis 18) together with a plan-certificate inclusion proof in a plan transparency log (PTL).[58]

1 Introduction

The companion paper Operational A (Schedule-only retries for AnonDHT) argues that SRPwrappers arrest trace leakage from retries, hedges, and negotiation.[60] In Operational A, the residual risk of a real implementation is captured by an explicit *schedule-only deviation* parameter $\varepsilon(W)$ that adds directly to the trace-privacy bound (while any attempt-level error composes linearly in the public slot budget). This paper addresses the next question: *how do we keep $\varepsilon(W)$ small over time?*

Two failure modes are common:

- **Untracked plan drift.** A deployment silently changes its schedule skeleton (slot counts, lane order, dummy fill policy).
- **In-implementation deviation.** The plan stays constant, but the implementation deviates (bugs, timeouts, pool interactions), leaking via micro-structure in the trace.

We propose a concrete operational response: *commit, log, and monitor*. This is inspired by transparency systems in security (Certificate Transparency, CT [2, 3]) and in the software supply chain (Sigstore/Rekor [9, 10]), and by safe anytime-valid inference for continuous monitoring [20, 24, 25, 31].

Contributions.

- **Proof-carrying plans:** a deployment provides a verifiable commitment (a “plan certificate”) binding code/config to an SRPschedule skeleton.
- **Plan transparency logs (PTL):** a CT-style append-only log of plan certificates, enabling external auditing and detecting equivocation.
- **Anytime-valid canaries:** e-process based monitoring that remains valid under optional stopping and continuous peeking.
- **Executable artifacts:** minimal tools to generate/verify plan certificates, append and audit a PTL, and run a trace-audit CLI.

2 Threat Model and Interfaces

We assume an operator may be honest-but-fallible; an attacker may be external (probing) or internal (a compromised component). We want to detect: (i) *misconfiguration* (wrong plan), (ii) *deviation* (runtime doesn’t match the plan), and (iii) *regressions* over time.

Interface: the plan skeleton. A SRPplan is a public schedule/lane description (slots, lanes, timing class) together with pinned parameters (timeouts, retry budgets). The plan is not a secret; the point is that SRPrequires that secrets not influence externally visible traces.

Plan identifiers: `plan_id` and `plan_spec_id`. To align with the receipt schema and drift taxonomy, we use the series’ replay-plan identifier pair.[57, 58] A schedule-only deployment names its plan family by a short `plan_id` (e.g., `plan.srp.traceaudit.v1`) and commits to its semantics by publishing a `plan_spec` record. The `plan_spec` includes the public schedule/lane skeleton,

pinned parameters (timeouts, retry budgets), and any monitored feature family/thresholds used by this paper’s canaries. Define the digest-bound identifier

$$\text{plan_spec_id} := \text{sha256}(\text{canon}(\text{plan_spec_core})),$$

where `plan_spec_core` is the minimal semantic core and `canon` is a deterministic canonicalization.[58] We attach `(plan_id, plan_spec_id)` to traces and audit artifacts so “same plan id, different plan” cannot hide off-surface.

Binding the interface and the window. To keep the operational objects aligned with the series’ anonymity claims, the plan object should carry `exposure_nf_id` (ENF-ID) for the audited trace projection and `tw_id` (TW-ID) for the monitoring window. Including these ids (and their canonical field instantiations) in the hashed plan inputs prevents a deployment from “keeping the same `plan_id`” while silently changing what it logs or what horizon it claims to monitor.[55, 56]

The plan certificate then binds `(plan_id, plan_spec_id)` to build metadata (e.g., compiler ID, code hash, and config hash) and is what gets recorded in the PTL. The explicit domain separator, delimiter, and canonicalization rules are important: auditors should be able to recompute `plan_spec_id` byte-for-byte from published plan inputs. For JSON-structured inputs, we recommend using a standard canonicalization such as JCS (RFC 8785) to avoid ambiguous encodings; our reference tooling uses a restricted, dependency-free canonicalization suitable for the demo plan types.[17]

A concrete bridge to $\varepsilon(W)$ in Operational A: monitoring a feature family. Operational A’s end-to-end wrapper deviation $\varepsilon(W)$ is a total-variation quantity over full traces. Operationally, we pin and monitor a finite family $\mathcal{F}$ of trace features (predicates, counters, lane-use bits) and target the induced deviation (an integral probability metric over $\mathcal{F}$)[18]

$$\varepsilon^{\mathcal{F}}(W) := \sup_{\sigma} \sup_{s_0, s_1} \sup_{f \in \mathcal{F}} |\mathbb{E}[f(\text{Trace}_{s_0})] - \mathbb{E}[f(\text{Trace}_{s_1})]|.$$

If $\mathcal{F}$ is rich enough to separate trace distributions then $\varepsilon^{\mathcal{F}}$ tracks the true ε ; otherwise it is a conservative, feature-scoped guarantee. Crucially, we treat $\mathcal{F}$ and its alerting budget as part of the plan by encoding them in the plan object (e.g., as additional `pinned_params` fields). Since `plan_spec_id` hashes both the plan skeleton and `pinned_params`, this makes the monitored feature family and its thresholds part of the same identifier that traces and auditors use.

Proposition 1 (Feature drift vs. total variation (sanity bound)). *Fix a public schedule σ and let P, Q be the resulting trace distributions under two secrets. If every $f \in \mathcal{F}$ takes values in $[-1, 1]$, then*

$$\sup_{f \in \mathcal{F}} |\mathbb{E}_P[f] - \mathbb{E}_Q[f]| \leq 2 \text{TV}(P, Q).$$

In particular, any monitored feature drift lower-bounds the true end-to-end deviation: if an audit finds $|\mathbb{E}_P[f] - \mathbb{E}_Q[f]| \geq \eta$ for some bounded f , then necessarily $\text{TV}(P, Q) \geq \eta/2$.

This bound is one-directional: controlling a small feature family does not by itself certify small $\varepsilon(W)$ unless $\mathcal{F}$ is sufficiently expressive.

How to choose $\mathcal{F}$ (practical recipe). Pick a small feature family that covers the dominant ways a wrapper can leak. In practice: (i) *hard conformance predicates* (slot budget exceeded, illegal lane transitions, missing dummy fills), (ii) *negotiation/fallback bits* (e.g., upgrade lane vs. fallback lane), (iii) *coarse distributional summaries* (histograms of success slot index, retry counts, abort reason buckets), and (iv) optionally a *nonparametric IPM feature* such as the maximum mean discrepancy (MMD) computed on low-dimensional trace embeddings.[36, 18] We commit $\mathcal{F}$ and its alerting budget in the plan so that auditors can later verify what was monitored. An example MMD-based feature demo is included in `evidence/mmd_feature_demo.py`. More sophisticated anytime-valid two-sample monitoring methods (including sequential and learned-feature variants) can be substituted when appropriate.[32] For instance, e-values can be constructed for classifier-based two-sample tests (E-C2ST) and for sequential nonparametric two-sample testing by betting, providing additional options when simple handcrafted features are insufficient.[33, 34]

3 Proof-Carrying Plans

A *plan certificate* is a signed commitment to:

1. the plan skeleton (slots, lanes, timing class),
2. pinned parameters (timeouts, retry budgets),
3. the code/config identity that enforces the plan (hashes, build metadata).

Signing and verification (practical note). In production, the plan certificate should be signed by a deployment identity key (e.g., a build/release service key or an operator key), so third parties can verify that the deployment is actually bound to the published plan. For interoperability with supply-chain tooling, the certificate can be packaged as a DSSE envelope (a lightweight signature wrapper used broadly for attestations), which reduces reliance on ad-hoc canonicalization rules at the signature layer.[12, 14] Our reference tooling signs the canonicalized certificate fields (excluding the signature itself) with an explicit domain separator using Ed25519, and the PTL treats the serialized certificate as the leaf payload.

Auditor checklist (one page).

Check	Evidence artifact
Plan pinned?	Plan certificate (signature) over (<code>plan_id</code> , <code>plan_spec_id</code>) and policy inputs
Discoverable?	PTL inclusion proof for the certificate + latest checkpoint
Append-only?	PTL consistency proof between checkpoints (or per-shard checkpoint chain)
No split view?	Witness-cosigned checkpoints (or gossip/witness monitors)
Runtime matches plan?	Trace contract checks (slot/lane invariants) + canary status

This follows the “proof-carrying” spirit [1]: a deployment carries a compact object that lets auditors verify key properties quickly.

Trace linkage. Each trace record includes the `plan_spec_id` it claims to follow. The trace-audit CLI first checks that `plan_spec_id` matches the intended plan, then validates per-epoch conformance. Figure 1 shows the tool in action.

Figure omitted in source-only archive.
(Plot/diagram and scripts available on request.)

Figure 1: Trace-audit CLI (from the research archive): traces are validated against a committed plan identifier and monitored for deviations.

Figure omitted in source-only archive.
(Plot/diagram and scripts available on request.)

Figure 2: Toy plan transparency log (PTL) demo: append-only Merkle structure enables inclusion/consistency proofs, as in CT.

4 Plan Transparency Log (PTL)

Plan certificates are more useful if they are publicly *discoverable* and *non-equivocating*. We follow the standard transparency pattern: an append-only Merkle tree log as in Certificate Transparency (CT) [2, 3]. Transparency logs are now largely off the shelf: Certificate Transparency provides a widely deployed Merkle-tree log interface (checkpoints, inclusion proofs, and consistency proofs), and the same pattern is reused for non-PKI attestations (e.g., Sigstore-style transparency logs). Our PTL reuses exactly these interfaces; sharding/tiling and witness cosigning change only how proofs are fetched, not what an auditor must check.[2, 3, 10]

Our toy PTL is a single Merkle tree to keep the paper compact; sharding/tiling does not change the auditor checklist, only how proofs are fetched.[6]

Definition 1 (PTL interface (checkpoint, inclusion, consistency)). *A PTL maintains an append-only Merkle tree over submitted plan certificates (leaf payloads) and exposes: (i) signed checkpoints committing to tree size n and root r (optionally witness-cosigned), (ii) inclusion proofs showing a given leaf is present in the tree at a checkpoint size, and (iii) consistency proofs showing that a later checkpoint extends an earlier checkpoint without rewriting prior leaves.*

Proposition 2 (Append-only auditability (informal)). *If a plan certificate verifies and has a valid inclusion proof at checkpoint cp_2 , and auditors also verify a consistency proof from an earlier checkpoint cp_1 to cp_2 , then the log view at cp_2 is an append-only extension of the view at cp_1 (no fork that rewrites history without being detected).*

What auditors check. Given a plan certificate with `plan_spec_id`, auditors verify: (i) a valid signature, (ii) an inclusion proof for the certificate in the PTL, and (iii) a consistency proof between PTL checkpoints. This detects silent plan drift and supports third-party review. Figure 2 shows a toy-but-executable PTL.

Witnessing / cosigning. Transparency logs can still equivocate if clients do not gossip. Witness cosigning strengthens logs by proactively recording checkpoints with independent witnesses, as advocated by CoSi [19] and formalized in modern tlog cosignature/witness interfaces.[44, 45] Operational efforts package witnesses as plug-and-play components (e.g., ArmoredWitness) and publish configuration guidance for deploying witness sets; some operators also publish concrete witness endpoints and conformance statements.[8, 51] Figure 3 illustrates a witness-cosigned checkpoint flow.

Figure omitted in source-only archive.
(Plot/diagram and scripts available on request.)

Figure 3: Witness cosigning for PTL checkpoints (toy demo): witness-signed checkpoints reduce the risk of undetected equivocation.

Figure omitted in source-only archive.
(Plot/diagram and scripts available on request.)

Figure 4: Optional stopping intuition (from the research archive): classical p-values break under repeated peeking; e-values/e-processes remain valid at all stopping times.

5 Anytime-Valid Canary Monitoring

Auditing detects *what was committed*; monitoring detects *what is happening*.

Why anytime-valid? Operational monitoring involves continuous peeking (dashboards, alerts). Classical p-values inflate false positive rates under optional stopping. E-values and e-processes are designed for time-uniform validity [20, 24, 25]. Figure 4 visualizes the difference.

Where do the samples come from? (canary probes) To monitor a feature family $\mathcal{F}$ in a way that matches Operational A’s “two secrets” framing, operators can run *randomized canary sessions*. In each epoch (or test window), choose a canary label $C_t \in \{0, 1\}$ using public randomness, execute an otherwise plan-identical canary request from a small set of pinned canary inputs, and log the plan-defined feature vector X_t of the observed trace. Under end-to-end SRP, X_t should be independent of C_t (conditioned on the public plan), so any detected dependence is actionable evidence of drift and directly lower-bounds the feature-scoped deviation $\varepsilon^{\mathcal{F}}(W)$. Our lane-switch regression toy below can be read as this setting when X_t is the single binary feature “fallback lane used.”

Note on thresholds. The default rejection threshold $1/\alpha$ (from Markov/Ville) is intentionally conservative; under additional, empirically checkable distributional assumptions on e-values, the threshold can sometimes be lowered while retaining Type I control, improving detection power.[27]

Theorem 1 (Time-uniform false-alarm control (Ville/Doob maximal inequality, informal)). *Let $(E_t)_{t \geq 0}$ be a nonnegative e-process under H_0 , i.e., a nonnegative supermartingale with $E_0 = 1$. Define a stopping rule $\tau := \inf\{t : E_t \geq 1/\alpha\}$. Then $\Pr_{H_0}[\tau < \infty] \leq \alpha$. [21, 25, 22, 26, 35]*

Sanity check. The accompanying evidence scripts include an empirical H_0 false-alarm check over finite horizons (not plotted here).

A concrete regression: lane-switch / downgrade. A common SRP failure mode is negotiation fallback becoming data-dependent (or plan-inconsistent), turning “fallback used” into an amplifiable bit. We include a minimal synthetic experiment: a single binary feature per epoch (fallback lane used) with baseline probability p_0 . Under regression, the probability increases by Δ . Using a likelihood-ratio e-process, we estimate detection times vs. Δ while preserving time-uniform false-alarm control. For a single binary feature $X_t \in \{0, 1\}$ with null rate p_0 and an alternative rate

Figure omitted in source-only archive.
 (Plot/diagram and scripts available on request.)

Figure 5: Synthetic lane-switch regression detection (script and CSV in **evidence/**): larger down-grade shifts are detected faster; the $\log_2(1/\alpha)/\text{KL}$ curve provides a useful sizing heuristic.

Figure omitted in source-only archive.
 (Plot/diagram and scripts available on request.)

Figure 6: Audit playbook flowchart (from the research archive).

$p_1 > p_0$, the canonical per-epoch likelihood ratio is

$$L_t(p_1) = \left(\frac{p_1}{p_0}\right)^{X_t} \left(\frac{1-p_1}{1-p_0}\right)^{1-X_t}, \quad E_T(p_1) = \prod_{t=1}^T L_t(p_1),$$

which is an e-process under H_0 ; in practice we use a simple mixture over a small grid of p_1 values to avoid picking a single alternative. Figure 5 also overlays a back-of-the-envelope approximation $\log_2(1/\alpha)/\text{KL}(p_1||p_0)$.

Multi-feature monitoring (omitted for space). The research archive includes a multifeature scaling experiment showing how combining trace features can improve detection speed while preserving time-uniform guarantees.

6 Sizing (brief)

There is an inherent race: adversaries can exploit deviations quickly, while auditors need enough samples to detect them. The research archive includes an “attack-vs-audit” sizing worksheet and plot; we omit the figure here for space.

7 Operational playbook (condensed)

A practical playbook:

1. **Pin** a plan skeleton and produce a plan certificate at build/rollout time.
2. **Append** the certificate to the PTL (obtain inclusion proof; record checkpoint).
3. **Monitor** traces against the plan (contract checks) and run anytime-valid canaries for drift.
4. **Respond** to alarms: freeze rollouts, compare deployed `plan_spec_id` to expected, and publish incident notes.

8 Related Work (selected)

Certificate Transparency [2, 3] is the canonical transparency-log design for PKI. Key transparency systems (e.g., CONIKS and Google Key Transparency) apply similar append-only log and monitoring ideas to public key directories and account keys.[38, 39] Studies of CT in practice emphasize the

importance of monitor diversity, completeness, and anti-equivocation techniques [29, 30, 28]. Sigstore applies similar ideas to software artifacts via a public log (Rekor) [9, 10]. Related transparency frameworks are being explored in other domains (e.g., confidential computing transparency for TEEs) [37]. Supply-chain attestation frameworks (e.g., in-toto and SLSA) and emerging IETF work on SCITT likewise center on signed statements plus transparency/receipts (including COSE Receipts with Merkle inclusion/consistency proofs), making PTL-style logs a natural fit for plan certificates.[11, 13, 15, 49, 50, 53] Witness cosigning (CoSi) strengthens transparency services by proactively preventing undetected equivocation [19]. For the monitoring layer, safe anytime-valid inference (e-values/e-processes) enables optional-stopping-robust alerting [20, 24, 25].

9 Limitations (brief)

This paper focuses on making trace-privacy *auditable* and *monitorable*, not on hiding the fact that a plan exists. Because plans and checkpoints are public, they can reveal operational metadata (e.g., which schedule families a service supports); if that is sensitive, deployments can aggregate or delay publication, or publish only a coarse plan family while keeping fine-grained parameters internal. Finally, the monitoring layer is only as good as the committed feature family $\mathcal{F}$: if drift manifests outside $\mathcal{F}$ it may go undetected until $\mathcal{F}$ is expanded or the plan is revised.

10 Conclusion

Trace privacy needs governance. Proof-carrying plans, transparency logs, and anytime-valid monitoring provide a practical path from a paper design to an auditable, regression-resistant deployment discipline.

11 Takeaways

- **Pin the plan:** a deployment exports a `plan_spec_id` and a signed plan certificate binding `plan_spec_id` to build/config hashes; the certificate is appended to a PTL.
- **Make drift checkable:** traces carry `plan_spec_id`; auditors verify conformance and publish an inclusion proof plus an epoch receipt.
- **Monitor under peeking:** e-process/e-value receipts bound feature-level drift $\varepsilon^{\mathcal{F}}(W)$ under optional stopping, giving an operational proxy for Operational A’s schedule-only deviation.
- **Artifacts (on request):** reference tooling (`tools/`) and demos (`evidence/`) are available on request under the series artifact policy.[59]

References

- [1] George C. Necula. Proof-Carrying Code. Proceedings of the 24th ACM SIGPLAN–SIGACT Symposium on Principles of Programming Languages (POPL), 106–119, 1997. DOI: 10.1145/263699.263712.
- [2] Ben Laurie and Adam Langley and Emilia Kasper. Certificate Transparency. IETF, 2013. DOI: 10.17487/RFC6962. <https://www.rfc-editor.org/rfc/rfc6962.html>
- [3] Ben Laurie and Emilia Messeri and Rob Stradling. Certificate Transparency Version 2.0. IETF, 2021. DOI: 10.17487/RFC9162. <https://www.rfc-editor.org/rfc/rfc9162.html>

- [4] Google Trillian Project. Trillian: General Transparency (Log Mode Overview). Project documentation, 2026. States Trillian is in maintenance mode and recommends Tessera for new tiled transparency logs. Accessed 2026-02-22. <https://google.github.io/trillian/>
- [5] transparency-dev. Tessera: Go library for building tile-based transparency logs (tlogs). GitHub repository, 2024. Describes Tessera as a tile-based successor to Trillian v1 and includes witness and Static CT API support. Accessed 2026-02-21. <https://github.com/transparency-dev/tessera>
- [6] transparency.dev. Tile-Based Transparency Logs. Technical article, 2024. Overview of tiled log APIs (checkpoints, tiles, and proof distribution) and operational motivations. Accessed 2026-02-21. <https://transparency.dev/articles/tile-based-logs/>
- [7] transparency-dev. TesseraCT (**tesseract**): Static CT API server built on Tessera. GitHub repository, 2025. Implements the Static CT API on top of Tessera for production-grade CT logs. Accessed 2026-02-21. <https://github.com/transparency-dev/tesseract>
- [8] transparency.dev. ArmoredWitness: A plug-and-play witness solution. GitHub repository, 2024. Project intended to kick-start a cross-ecosystem witness network and reduce split-view risk. Accessed 2026-02-21. <https://github.com/transparency-dev/armored-witness>
- [9] Zachary Newman and others. Sigstore: Software Signing for Everybody. ACM Conference on Computer and Communications Security (CCS), 2022. DOI: 10.1145/3548606.3560596.
- [10] Sigstore Project. Rekor Transparency Log: Overview. Documentation, 2026. <https://docs.sigstore.dev/logging/overview/>
- [11] Torres-Arias, Santiago and others. in-toto: Providing farm-to-table guarantees for bits and bytes. 28th USENIX Security Symposium (USENIX Security 19), 2019. <https://www.usenix.org/system/files/sec19-torres-arias.pdf>
- [12] Secure Systems Lab. DSSE: Dead Simple Signing Envelope (specification). GitHub repository (spec), 2026. Defines a lightweight envelope format for signing arbitrary payloads; used by multiple supply-chain attestation systems. Accessed 2026-02-22. <https://github.com/secure-systems-lab/dsse>
- [13] OpenSSF / SLSA. SLSA Specification (v1.2). Specification, 2026. Approved specification; accessed 2026-02-22. <https://slsa.dev/spec/v1.2/>
- [14] OpenSSF / SLSA. SLSA v1.2: Software Attestations (recommended envelope: DSSE). Specification page, 2026. Defines a recommended suite of attestation building blocks and lists DSSE as the recommended envelope. Accessed 2026-02-22. <https://slsa.dev/spec/v1.2/attestation-model>
- [15] Birkholz, Henk and others. An Architecture for Trustworthy and Transparent Digital Supply Chains (SCITT). Internet-Draft, **draft-ietf-scitt-architecture-22**, 2025. Expires 13 April 2026; accessed 2026-02-22. <https://datatracker.ietf.org/doc/draft-ietf-scitt-architecture/>
- [16] Sigstore Project. Rekor v2 GA: Cheaper to run, simpler to maintain. Blog post, 2025. <https://blog.sigstore.dev/rekor-v2-ga/>
- [17] Rundgren, Anders. JSON Canonicalization Scheme (JCS). IETF, 2020. DOI: 10.17487/RFC8785. <https://www.rfc-editor.org/rfc/rfc8785>
- [18] Sriperumbudur, Bharath K. and Fukumizu, Kenji and Gretton, Arthur and Sch"olkopf, Bernhard and Lanckriet, Gert R. G.. On the Empirical Estimation of Integral Probability Metrics. *Electronic Journal of Statistics*, 6, 1550–1599, 2012. DOI: 10.1214/12-EJS722. <https://projecteuclid.org/journals/electronic-journal-of-statistics/volume-6/issue-none/On-the-empirical-estimation-of-integral-probability-metrics/10.1214/12-EJS722.full>

- [19] Ewa Syta and Iulia Tamas and Dylan Visser and David Isaac Wolinsky and Philipp Jovanovic and Linus Gasser and Nicolas Gailly and Ismail Khoffi and Bryan Ford. Keeping Authorities “Honest or Bust” with Decentralized Witness Cosigning. IEEE Symposium on Security and Privacy (S&P), 2016. DOI: 10.1109/SP.2016.38. <https://www.davidwolinsky.com/site/files/syta16sp.pdf>
- [20] Aaditya Ramdas and Peter Grünwald and Vladimir Vovk and Glenn Shafer. Game-Theoretic Statistics and Safe Anytime-Valid Inference. *Statistical Science*, 38(4), 2023. DOI: 10.1214/23-STS894. <https://projecteuclid.org/journals/statistical-science/volume-38/issue-4/Game-Theoretic-Statistics-and-Safe-Anytime-Valid-Inference/10.1214/23-STS894.full>
- [21] Ville, Jean. Étude critique de la notion de collectif. 1939. Introduces Ville’s martingale inequality underlying time-uniform bounds; scanned thesis/book hosted by Numdam. Accessed 2026-02-22. https://www.numdam.org/item/THESE_1939__218__1_0.pdf
- [22] Howard, Steven R. and Ramdas, Aaditya and McAuliffe, Jon and Sekhon, Jasjeet. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2), 2021. DOI: 10.1214/20-AOS1991. <https://projecteuclid.org/journals/annals-of-statistics/volume-49/issue-2/Time-uniform-nonparametric-nonasymptotic-confidence-sequences/10.1214/20-AOS1991.full>
- [23] Let’s Encrypt. End of Life Plan for RFC 6962 Certificate Transparency Logs. Blog post, 2025. <https://letsencrypt.org/2025/08/14/rfc-6962-logs-eol>
- [24] Shafer, Glenn. Testing by Betting: A Strategy for Statistical and Scientific Communication. *Journal of the Royal Statistical Society: Series A*, 2021. https://glennshafer.com/assets/downloads/articles/article104_jrss_shafer-testingbybetting-with-discussion-response.pdf
- [25] Ramdas, Aaditya and others. Hypothesis Testing with E-values. Monograph (draft), 2025. <https://www.stat.cmu.edu/~aramdas/ebook-final.pdf>
- [26] Ruf, Johannes and Larsson, Martin and Koolen, Wouter M. and Ramdas, Aaditya. A Composite Generalization of Ville’s Martingale Theorem Using e-Processes. *Electronic Journal of Probability*, 28, 1–21, 2023. DOI: 10.1214/23-EJP1019. <https://projecteuclid.org/journals/electronic-journal-of-probability/volume-28/issue-none/A-composite-generalization-of-Villes-martingale-theorem-using-e-processes/10.1214/23-EJP1019.full>
- [27] Blier-Wong, Christopher and Wang, Ruodu. Improved Thresholds for E-values. Preprint, 2026. Discusses improved rejection thresholds for e-tests/e-processes under additional distributional assumptions; dated Feb 7, 2026. Accessed 2026-02-22. <https://www.math.uwaterloo.ca/~wang/papers/2026BlierWong-Wang-AOS.pdf>
- [28] Rasmus Dahlberg and Tobias Pulls and Roel Peeters. Aggregation-Based Certificate Transparency Gossip. arXiv:1806.08817, 2018. <https://arxiv.org/abs/1806.08817>
- [29] Bingyu Li and Chuan Qin and Jianjun Chen and others. Certificate Transparency in the Wild. ACM Conference on Computer and Communications Security (CCS), 2019. <https://www.inforsec.org/wp/wp-content/uploads/2020/04/CCS2019-Li.pdf>
- [30] Aozhuo Sun and Jinqiang Lin and Wei Wang and others. The Public Inspections on Third-party Monitors. Network and Distributed System Security Symposium (NDSS), 2024. <https://www.ndss-symposium.org/wp-content/uploads/2024-834-paper.pdf>
- [31] Alexander Ly and Udo Boehm and Peter Grünwald and Aaditya Ramdas and Don van Ravenzwaaij. A Tutorial on Safe Anytime-Valid Inference: Practical Maximally Flexible Sampling Designs for Experiments Based on e-Values. Preprint, 2025. <https://www.alexander-ly.com/wp-content/uploads/2025/08/saviTutorial.pdf>

- [32] Pandeva, Teodora and Forré, Patrick and Ramdas, Aaditya and Shekhar, Shubhanshu. Deep Anytime-Valid Hypothesis Testing. Proceedings of The 27th International Conference on Artificial Intelligence and Statistics (AISTATS), 238, 622–630, 2024. <https://proceedings.mlr.press/v238/pandeva24a.html>
- [33] Pandeva, Teodora and Bakker, Tim and Naesseth, Christian A. and Forré, Patrick. E-Valuating Classifier Two-Sample Tests. Transactions on Machine Learning Research (TMLR), 2024. Introduces e-values for classifier two-sample tests (E-C2ST) suitable for anytime-valid sequential testing. <https://openreview.net/pdf?id=dwFRov8xhr>
- [34] Podkopaev, Aleksandr and Ramdas, Aaditya. Sequential Predictive Two-Sample and Independence Testing. Advances in Neural Information Processing Systems (NeurIPS), 2023. Nonparametric sequential two-sample testing by betting with anytime-valid guarantees. https://proceedings.neurips.cc/paper_files/paper/2023/file/a72b207734d6112f6b47447e46be40e9-Paper-Conference.pdf
- [35] Koolen, Wouter M. and P’erez-Ortiz, Muriel F. and Lardy, Tyron. A Generalisation of Ville’s Inequality to Monotonic Lower Bounds and Thresholds. Statistics & Probability Letters, 229, 110577, 2026. DOI: 10.1016/j.spl.2025.110577. <https://ir.cwi.nl/pub/35761/35761.pdf>
- [36] Gretton, Arthur and Borgwardt, Karsten M. and Rasch, Malte J. and Schölkopf, Bernhard and Smola, Alexander J.. A Kernel Two-Sample Test. Journal of Machine Learning Research, 13, 723–773, 2012. <https://www.jmlr.org/papers/volume13/gretton12a/gretton12a.pdf>
- [37] Kocaoğullar, Ceren and Marjanov, Tina and Petrov, Ivan and Laurie, Ben and Cutter, Al and Kern, Christoph and Hutchings, Alice and Beresford, Alastair R.. Confidential Computing Transparency. 2024. arXiv:2409.03720 <https://arxiv.org/abs/2409.03720>
- [38] Melara, Marc S. and Blankstein, Aaron and Bonneau, Joseph and Felten, Edward W. and Freedman, Michael J.. CONIKS: Bringing Key Transparency to End Users. Proceedings of the 24th USENIX Security Symposium, 2015. <https://www.usenix.org/system/files/conference/usenixsecurity15/sec15-paper-melara.pdf>
- [39] Google. Key Transparency (open-source implementation). GitHub repository, 2024. <https://github.com/google/keytransparency>
- [40] C2SP. Tiled Transparency Logs (tlog-tiles). Community specification, 2025. Specifies an HTTP API for fetching checkpoints, tiles, and entries of a tiled transparency log. Accessed 2026-02-22. <https://c2sp.org/tlog-tiles>
- [41] C2SP. Static Certificate Transparency API (static-ct-api). Community specification, 2025. Specifies the public endpoints of the Static CT API used by next-generation CT logs. Accessed 2026-02-22. <https://c2sp.org/static-ct-api>
- [42] C2SP: tlog-checkpoint. Community Cryptography Specification Project. Accessed 2026-02-22 <https://c2sp.org/tlog-checkpoint>
- [43] C2SP: tlog-proof. Community Cryptography Specification Project. Accessed 2026-02-22 <https://c2sp.org/tlog-proof>
- [44] C2SP: tlog-cosignature. Community Cryptography Specification Project. Accessed 2026-02-22 <https://c2sp.org/tlog-cosignature>
- [45] C2SP: tlog-witness. Community Cryptography Specification Project. Accessed 2026-02-22 <https://c2sp.org/tlog-witness>
- [46] Google Chrome. Chrome Certificate Transparency Policy. Web page, 2026. Accessed 2026-02-22; includes the transition rule that before April 15, 2026 at least one SCT must be from an RFC6962-compliant log. https://googlechrome.github.io/CertificateTransparency/ct_policy.html

- [47] Let's Encrypt. Certificate Transparency (CT) Logs. Documentation page, 2026. Summarizes RFC 6962 log EOL status and shutdown timeline; updated 2026-01-04. <https://letsencrypt.org/docs/ct-logs/>
- [48] Filippo Valsorda. Building a Transparent Keyserver. Technical article, 2025. Step-by-step integration of a tiled transparency log (Tessera) and witness cosigning (Witness Network) into a centralized service. Accessed 2026-02-22. <https://words.filippo.io/keyserver-tlog/>
- [49] IETF SCITT Working Group. Supply Chain Integrity, Transparency, and Trust (SCITT) Working Group. IETF Datatracker (group page), 2026. Accessed 2026-02-22. <https://datatracker.ietf.org/group/scitt/about/>
- [50] IETF SCITT Working Group. COSE Receipts with CCF. Internet-Draft, draft-ietf-scitt-receipts-ccf-profile-00, 2026. Accessed 2026-02-22. <https://datatracker.ietf.org/doc/draft-ietf-scitt-receipts-ccf-profile/>
- [51] Transparency.dev (TrustFabric team). TrustFabric Witnesses. Project page, 2025. Lists witnesses and notes conformance to C2SP tlog-witness and related specs. Accessed 2026-02-22. <https://transparency.dev/witnesses/>
- [52] Benjamin, David and others. Merkle Tree Certificates. Internet-Draft, texttttdraft-ietf-plants-merkle-tree-certs-01, 2026. IETF PLaNTS draft extending RFC 9162 with subtree proof objects and cosigner roles. Accessed 2026-02-22. <https://datatracker.ietf.org/doc/draft-ietf-plants-merkle-tree-certs/>
- [53] Orie Steele and Henk Birkholz and Amaury Delignat-Lavaud and Cedric Fournet. COSE (CBOR Object Signing and Encryption) Receipts. Internet-Draft, draft-ietf-cose-merkle-tree-proofs-18, 2025. Accessed 2026-02-22. <https://datatracker.ietf.org/doc/draft-ietf-cose-merkle-tree-proofs/>
- [54] Anonymous. Transparency Primitives for Proof-Carrying Anonymity Claims: Beacons, VRFs, and Append-Only Logs. Series synthesis note (Synthesis 9), The-One-True-Archive (2026).
- [55] Anonymous. Trace Interfaces for Anonymous DHTs: Observation Tiers, Committee-Contact Traces, and Exposure Normal Forms. Series synthesis note (Synthesis 3), 2026. (This archive.)
- [56] Anonymous. Threat Models and Repetition Windows for Anonymous DHTs: A Short Declaration Checklist for Referee-Grade Budget Claims. Series synthesis note (Synthesis 4), 2026. (This archive.)
- [57] Anonymous. Receipt Line Items for Anonymous-DHT Privacy Claims: Minimal Tuple Schema and Drift Semantics. Series synthesis note (Synthesis 12), 2026. (This archive.)
- [58] Anonymous. Digest-Bound Replay Plans and State Declarations: Equivalence, Minimality, and Change Control. Series synthesis note (Synthesis 18), 2026. (This archive.)
- [59] Anonymous. Artifact policy and supporting materials for the anonymity/AnonDHT series. 2026. See synthesis paper: Artifact policy (this archive). Artifacts, tooling, certificates, and scripts are available on request as a single archive.
- [60] Anonymous DHT Series (draft). *Schedule-Only Retries for AnonDHT: The SRP Wrapper Contract*. In this archive: Operational Series, Paper A.
- [61] Anonymous DHT Series (published). *Scheduling and Compiler Techniques for Metadata-Hiding Overlay Lookups*. [[2026.01.22 - Mathematics: Scheduling and Compiler Techniques for Metadata-Hiding Overlay Lookups]].
- [62] Anonymous DHT Series (published). *Stop-Time Padding Addendum, Max-Mixture Separations and Tail-Sign Deadline Normal Forms*. [[2026.01.26 - Mathematics: Stop-Time Padding Addendum, Max-Mixture Separations and Tail-Sign Deadline Normal Forms]].

- [63] Anonymous. Notation and Minimal Model for the One-True-Archive. Series synthesis note (Synthesis 8), 2026. (This archive.)